

Model and Posterior Used in the Demo

Source files: `demo_ess.m` and `elliptical_slice_step.m`

Model in the code

$$f \sim \mathcal{N}(0, K), \quad K = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}$$

$$y = hf + \varepsilon, \quad h = (1, 0), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2), \quad y = 1, \quad \sigma = 0.35.$$

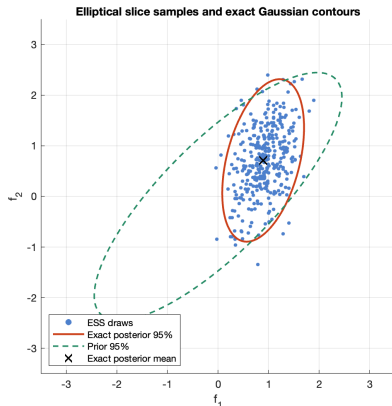
Posterior target

$$p(f | y) \propto L(f) \mathcal{N}(f; 0, K), \quad \log L(f) = -\frac{1}{2} \frac{(y - hf)^2}{\sigma^2}.$$

$$f | y \sim \mathcal{N}(m, C), \quad \begin{aligned} m &= Kh'(hKh' + \sigma^2)^{-1}y, \\ C &= K - Kh'(hKh' + \sigma^2)^{-1}hK. \end{aligned}$$

What the code does

- `demo_ess.m` builds a two-variable Gaussian example.
- `elliptical_slice_step.m` performs one Markov chain update.
- The demo checks the sampler against the exact Gaussian posterior.
- The final plot compares simulated draws, the exact posterior contour, and the prior contour.



Takeaway. The demo is small because all model-specific information enters through a covariance matrix and a log-likelihood function.

Step 1: reset and define the model

```
1  %DEMO_ESS Elliptical slice sampling in a two-variable Gaussian model.
2  % Run this script from the ESS folder. It needs only base MATLAB.
3
4  clear; clc; close all;
5  rng(32706);
6
7  % Prior:  $f \sim N(0, K)$ . Observe  $y = f_1 + \text{epsilon}$ ,
8  %  $\text{epsilon} \sim N(0, \text{sigma}^2)$ . The exact posterior is Gaussian here,
9  % so we can check the ESS output against known moments.
10 K = [1.0, 0.8; 0.8, 1.0];
11 cholK = chol(K, 'lower');
12 h = [1.0, 0.0];
13 y = 1.0;
14 sigma = 0.35;
```

- `rng(32706)` makes the simulation reproducible.
- K is the prior covariance of $f = (f_1, f_2)'$.
- The observation is $y = f_1 + \varepsilon$, with $\varepsilon \sim N(0, \sigma^2)$.
- `loglike` is the only likelihood object the ESS update needs.

Step 2: compute the exact posterior

```
1  
2  % Exact posterior moments for the linear-Gaussian example.  
3  denom = h * K * h' + sigma^2;  
4  m_exact = K * h' * (y / denom);
```

The example is deliberately linear-Gaussian, so the posterior is available in closed form:

$$C = K - Kh'(hKh' + \sigma^2)^{-1}hK,$$
$$m = Kh'(hKh' + \sigma^2)^{-1}y.$$

These exact moments are not needed by the sampler. They are used to verify the code.

Takeaway. The exact posterior is a benchmark for the lesson, not an input to ESS.

Step 3: run the Markov chain

```
1  % Draw a Markov chain. Discard the initial part of the chain.
2  nBurn = 1000;
3  nKeep = 6000;
4  samples = zeros(2, nKeep);
5  f = zeros(2, 1);
6  totalEval = 0;
7  for t = 1:(nBurn + nKeep)
8      [f, nEval] = elliptical_slice_step(f, cholK, loglike);
9      totalEval = totalEval + nEval;
10     if t > nBurn
11         samples(:, t - nBurn) = f;
12     end
13 end
14
```

1. Start the chain at $f = (0, 0)'$.
2. Run $nBurn + nKeep$ updates.
3. Store only draws after burn-in.
4. Count likelihood evaluations to measure computational effort.

Step 4: compare exact and simulated moments

```
1 m_sample = mean(samples, 2);
2 C_sample = cov(samples');
3 fprintf('Exact posterior mean:      [%7.4f, %7.4f]\n', m_exact);
4 fprintf('ESS sample mean:         [%7.4f, %7.4f]\n', m_sample);
5 fprintf('Exact posterior covariance:\n'); disp(C_exact);
6 fprintf('ESS sample covariance:\n'); disp(C_sample);
7 fprintf('Mean log-likelihood evaluations per update: %.2f\n', ...
8         totalEval / (nBurn + nKeep));
```

- The posterior mean is estimated by the average of stored draws.
- The posterior covariance is estimated with `cov(samples')`.
- The printed output lets students check convergence numerically before looking at the plot.

Step 5: prepare posterior and prior contours

```
1 [x1, x2] = meshgrid(linspace(-3.7, 3.7, 180));  
2 grid_points = [x1(:)'; x2(:)'];  
3 centered = grid_points - m_exact;  
4 d2_post = reshape(sum(centered .* (C_exact \ centered), 1), size(x1));  
5 d2_prior = reshape(sum(grid_points .* (K \ grid_points), 1), size(x1));  
6 level95 = 5.991; % Chi-square(2) 95% quantile, for a contour only.  
7  
8 figure('Color', 'w'); hold on;
```

$$d_p^2(f) = (f - m)' C^{-1} (f - m),$$

$$d_0^2(f) = f' K^{-1} f.$$

The value 5.991 is the 95 percent quantile of a chi-square distribution with two degrees of freedom.

Step 6a: draw the figure

```
1 h_draws = scatter(samples(1, idx), samples(2, idx), 12, ...  
2                   [0.3 0.5 0.8], 'filled');  
3 [~, h_post] = contour(x1, x2, d2_post, [level95 level95], ...  
4                   'Color', [0.8 0.25 0.1], 'LineWidth', 1.8);  
5 [~, h_prior] = contour(x1, x2, d2_prior, [level95 level95], ...  
6                   'Color', [0.15 0.55 0.4], 'LineStyle', '--', ...  
7                   'LineWidth', 1.5);  
8 h_mean = plot(m_exact(1), m_exact(2), 'kx', 'MarkerSize', 10, ...  
9               'LineWidth', 2);  
10 axis equal; xlim([-3.5, 3.5]); ylim([-3.5, 3.5]); grid on;  
11 xlabel('f_1'); ylabel('f_2');  
12 title('Elliptical slice samples and exact Gaussian contours');
```

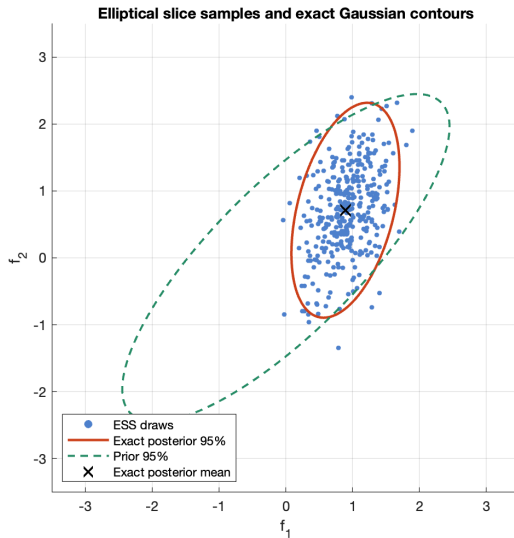
- A subset of draws is shown to avoid overplotting.
- The posterior contour is solid orange.
- The prior contour is green and dashed.
- The exact posterior mean is marked with a black cross.
- `exportgraphics` writes `ess_demo.png`.

Step 6b: label and export the figure

```
1 legend([h_draws, h_post, h_prior, h_mean], ...  
2        {'ESS draws', 'Exact posterior 95%', 'Prior 95%', ...  
3          'Exact posterior mean'}, 'Location', 'southwest');  
4  
5 exportgraphics(gcf, 'ess_demo.png', 'Resolution', 180);
```

- The exact posterior mean is marked with a black cross.
- The legend identifies the draws, prior, posterior, and mean.
- `exportgraphics` writes `ess_demo.png`.

The resulting picture



Inside one ESS update: input checks

```
1 function [f_new, nEval] = elliptical_slice_step(f, cholK, loglike)
2 %ELLIPTICAL_SLICE_STEP One ESS update for a zero-mean Gaussian prior.
3 % f_new = elliptical_slice_step(f, cholK, loglike) targets a density
4 % proportional to  $N(f; 0, K) * \exp(\loglike(f))$ , where cholK is a
5 % LOWER
6 % Cholesky factor:  $K = \text{cholK} * \text{cholK}'$ .
7 % nEval counts log-likelihood evaluations in this update.
8 % Reference: Murray, Adams and MacKay (2010), Elliptical slice
9 % sampling.
10
11 f = f(:);
12 n = numel(f);
13 if ~isequal(size(cholK), [n, n])
14     error('cholK must be an n-by-n lower Cholesky factor.');
```

- The state is forced into a column vector.
- cholK must be an $n \times n$ lower Cholesky factor.
- The current log-likelihood must be finite and scalar.
- nEval starts at one because the current state is evaluated once.

Inside one ESS update: slice and ellipse

```
1  logy = loglike_current + log(rand); % Slice level on the log
   scale
2
3  theta = 2 * pi * rand;
4  theta_min = theta - 2 * pi;
5  theta_max = theta;
6
7  while true
8      proposal = f * cos(theta) + nu * sin(theta);
```

- $\nu = \text{cholKz}$, with $z \sim N(0, I)$, is an independent prior draw.
- The log slice level is below the current log-likelihood.
- The first angle is random on $[0, 2\pi]$.
- The initial bracket has width 2π and contains angle zero.

Inside one ESS update: propose and accept

```
1      nEval = nEval + 1;
2      if ~isscalar(loglike_proposal) || isnan(loglike_proposal)
3          error('loglike must return a scalar value or -Inf.');
```

```
4      end
5
6      if loglike_proposal > logy
7          f_new = proposal;
8          return;
9      end
10
11      % The bracket always retains theta = 0, the current state.
12      if theta < 0
```

1. Propose
 $f(\theta) = f \cos \theta + \nu \sin \theta.$
2. Accept if the proposal is above the slice level.
3. Return immediately once an acceptable point is found.

Inside one ESS update: shrink and retry

```
1     else
2         theta_max = theta;
3     end
4     theta = theta_min + rand * (theta_max - theta_min);
5 end
6 end
```

1. A rejected angle cuts away one side of the bracket.
2. Angle zero remains in the bracket, so the current state is always available.
3. The loop draws from the smaller bracket until it finds an angle above the slice.

Why this update is convenient

- It uses the Gaussian prior geometry instead of a random-walk proposal.
- It requires no proposal variance or step-size tuning.
- The likelihood can be swapped by changing only the `loglike` function handle.
- The accepted point is always on the prior ellipse through the current state and an independent prior draw.

Takeaway. The sampler separates prior geometry from likelihood evaluation, which is why the implementation is compact.

Files produced by the demo

File	Role	Key idea
demo_ess.m	driver script	define model, run chain, plot checks
elliptical_slice_step.m	sampler step	slice level plus angular bracket shrinking
ess_demo.png	output figure	simulated draws vs exact contours
beamer.tex	lecture deck	step-by-step code walkthrough